AGENTSABHA

MASTER LLM BUILD PROMPT

Complete context · Architecture · Schema · Agent specs · Build order

Field	Value
Project	AgentSabha — एजेंट सभा
Tagline	543 AI Agents. One Parliament. A Billion Voices.
Mission	One autonomous AI agent per Lok Sabha constituency; collects citizen issues; performs every parliamentary function; powers civic media engine
Phase 1 goal	3 constituencies · 1 confirmed MP partner · 1 real parliamentary question filed · 1 citizen notified
Stack	Python 3.11 + FastAPI · PostgreSQL 16 + pgvector · Redis + Celery · Claude API · Next.js 14
Document version	v2.0 — Post governance/legal review · March 2026
Prompt target	Claude Code / GPT-4o / Gemini / any capable coding LLM

HOW TO USE THIS DOCUMENT

This document is the single source of truth for any LLM building AgentSabha. Read every section before writing a single line of code. Each section is a module — implement in order. Never skip ahead. If you are unsure about any decision, re-read the relevant section. If a section says DO NOT, that is a hard constraint. Do not rationalise around it.

> *HALLUCINATION PREVENTION RULE: Every technical decision in this document is grounded in the real system design. If something is not specified here, ask before implementing. Do not invent schema columns, API endpoints, agent behaviours, or business logic that are not in this document. When in doubt: document first, code second.*

SECTION 1 — WHAT YOU ARE BUILDING

Read this fully before touching any code.

AgentSabha deploys one autonomous AI agent per Lok Sabha constituency — 543 agents total. Each agent continuously collects citizen issues via WhatsApp, clusters them by severity and velocity, and produces real parliamentary instruments: questions, Zero Hour notices, debate speeches, and Private Members Bills. A national media engine converts the weekly data into a fully-automated podcast, Instagram Reels, and MP briefings — no human in the editorial loop.

The platform has two audiences running simultaneously:

- CITIZENS: submit local issues (road, water, power, health, schools) via WhatsApp in 22 Indian languages. They receive acknowledgment, rank updates, and notification when their issue reaches Parliament.
- OBSERVERS (MPs, journalists, government, corporates): consume structured intelligence derived from the citizen data — weekly MP briefs, journalist data APIs, government planning intelligence, financial risk scores.

1.1 The Core Loop — Build This First

> *citizen submits issue → AI clusters → MP gets draft question → MP files → citizen gets notified. That loop, proven once with a real MP, is the entire Phase 1.*

Step	Who	Action	Output	SLA
1	Citizen	WhatsApp message / voice note in any language	Raw message received	<5s to acknowledgment
2	Intake Agent	Transcribe → translate → extract fields → embed → store	Structured issue record in DB	<30s processing
3	Clustering Agent	BERTopic every 15 min → group similar issues → badge assignment	Updated cluster + Top 10 ledger	<15 min from submission
4	Question Draft Agent	Map cluster → ministry → draft Rule 32 question with citations	Draft question with sources	Daily at 08:00 IST
5	MP Brief Agent	Compile weekly PDF: top 5 issues + 5 draft questions	PDF + WhatsApp delivery	Every Friday 09:00 IST
6	Human MP	Review brief → edit if needed → file question in Parliament	Real parliamentary question filed	MP decision
7	Response Monitor	Poll parliamentary records for ministry response	Response captured + stored	Within 24h of response
8	Notification Agent	Identify triggering citizen → send WhatsApp notification	Citizen notified	<1h of response capture

1.2 Phase 1 Scope — What to Build NOW

> *Phase 1 = 3 constituencies ONLY. Do not build for 543. Do not build the media engine. Do not build financial APIs. Do not build the podcast pipeline. Focus exclusively on the core loop above.*

Build in Phase 1	Defer to Phase 2+
Intake Agent (WhatsApp + web form)	Podcast / YouTube / Instagram Reels pipeline
Clustering Agent (BERTopic / DBSCAN)	Zero Hour Agent, Debate Agent, Bill Drafting Agent
Question Hour Draft Agent	Financial sector intelligence API
MP Brief Generator (PDF + WhatsApp)	Election intelligence products
Minimal public dashboard (map + ledger + draft log)	Vidhan Sabha / international deployment
Citizen acknowledgment + notification	Speaker Agent (multi-agent coordination)
Aadhaar OTP constituency verification	Mobile native app (WhatsApp first)
Basic audit logging	Full 543-constituency deployment

SECTION 2 — HARD CONSTRAINTS (DO NOT VIOLATE)

> *These are not suggestions. Violating any of these constraints will either break the legal framework, destroy user trust, or create a security incident. Read each one carefully.*

2.1 Legal Constraints

- AADHAAR: Never store Aadhaar numbers. Store only the VID (Virtual ID) token. Never use Aadhaar data for commercial intelligence products. Constituency verification only.
- DPDP ACT 2023: Citizens must give explicit, granular consent for each purpose separately: (a) issue storage, (b) constituency mapping, (c) anonymised aggregation, (d) MP brief inclusion. Implement consent_flags column in citizens table.
- DATA LOCALISATION: All data must be stored on servers physically in India. No AWS us-east, no EU regions. AWS ap-south-1 (Mumbai) only.
- CONTENT LIABILITY: Every factual claim in every agent output must link to a primary source. The Fact Check gate is a HARD GATE — nothing publishes without it. No exceptions.
- BLOCKCHAIN: Anchor ONLY cryptographic hashes to Polygon — never the content itself. Content lives on your servers (mutable/deletable). Hash proves existence.
- PARLIAMENTARY OUTPUTS: In Phase 1 and Phase 2, ALL parliamentary outputs are DRAFTS FOR MP REVIEW. AgentSabha does not file directly. MPs file. AgentSabha drafts.

2.2 Data Privacy Constraints

- Tier 0 (Raw PII): Aadhaar VID + mobile_hash + constituency link. AES-256 encrypted at rest. NEVER in logs. NEVER in API responses. Internal access only with audit trail.
- Tier 1 (Issue content): Raw + translated text + embeddings. Internal only. Embeddings must never be exposed externally (deanonymisation risk).
- Tier 2 (Aggregated clusters): Issue cluster labels + counts + severity averages. Public dashboard only. Minimum cluster size of 50 before publishing.
- Tier 3 (Parliamentary drafts): Draft questions + Zero Hour notices. MP for their constituency only + public after MP filing.
- Tier 4 (Media outputs): Podcast, Reels, press packages. Fully public.
- Tier 5 (Audit logs): Agent action logs + fact-check reports. Fully public. These are the credibility asset.

2.3 Architecture Constraints

- NEVER use WidthType.PERCENTAGE in DOCX tables — use DXA only. (If generating documents.)
- NEVER use unicode bullets directly in DOCX — use LevelFormat.BULLET with numbering config.
- NEVER use a single API key for all MP briefs. Role-based access: each MP key can only access their constituency data.
- NEVER run clustering on production DB directly — use a read replica or a dedicated analytics connection.
- NEVER store raw WhatsApp payloads with PII in application logs. Strip before logging.
- NEVER proceed to Phase 2 until all Phase 1 exit gates are verified. The gates are binary — all 8 must be true.

2.4 Agent Behaviour Constraints

- Every factual claim in an agent output MUST link to a primary source. No source = claim removed. No exceptions.
- The Fact Check Agent is a HARD GATE in the media pipeline. Script cannot proceed with unverified claims.
- Question Hour Agent must cite specific Lok Sabha rule (Rule 32-55) in every question output.
- No agent may take a position on value-contested issues (religion, caste, security). Report distribution of opinion only.
- All agent actions must be logged in agent_logs with: agent_type, constituency_id, action, input_hash, output_hash, model_version, tokens_used, timestamp.
- Shadow mode: Any new agent or major prompt change must run in shadow mode (no real output) for 2 weeks before going live.

SECTION 3 — COMPLETE DATABASE SCHEMA

Implement exactly as specified. Do not add columns without documenting them here first. Column names are snake_case. All timestamps are timestamptz (UTC). All IDs are UUID unless specified.

3.1 Core Tables — implement in this order

constituencies (seed data — 543 rows)

```
id              SERIAL PRIMARY KEY  -- 1-543, matches Lok Sabha numbering name
TEXT NOT NULL state           TEXT NOT NULL region          TEXT  --
north/south/east/west/central/northeast population      INTEGER area_km2        DECIMAL
mp_name         TEXT mp_party        TEXT mp_party_govt   BOOLEAN  -- true = currently in
govt lat             DECIMAL lng             DECIMAL created_at      TIMESTAMPTZ DEFAULT
NOW()
```

citizens

```
id               UUID PRIMARY KEY DEFAULT gen_random_uuid() mobile_hash       TEXT NOT
NULL UNIQUE  -- SHA-256 of E.164 mobile number aadhaar_vid       TEXT  -- encrypted VID
token only. NEVER Aadhaar number constituency_id   INTEGER REFERENCES constituencies(id)
verified          BOOLEAN DEFAULT false consent_flags     JSONB  -- {issue_storage,
mapping, aggregation, mp_brief} age_verified      BOOLEAN DEFAULT false  -- under-18
blocked created_at        TIMESTAMPTZ DEFAULT NOW() last_active       TIMESTAMPTZ
```

issues

```
id               UUID PRIMARY KEY DEFAULT gen_random_uuid() citizen_id        UUID
REFERENCES citizens(id) constituency_id   INTEGER REFERENCES constituencies(id) raw_text
TEXT NOT NULL  -- original language translated_text   TEXT  -- English translation
source_language   TEXT  -- ISO 639-1 code source_channel    TEXT  -- whatsapp | web |
ivrs | sms issue_type        TEXT  -- road|water|power|health|education|employment|other
severity_score    DECIMAL(3,1)  -- 1.0-10.0, LLM-assigned urgency_flag      BOOLEAN
DEFAULT false location_district TEXT location_ward     TEXT affected_estimate INTEGER  --
LLM estimate of affected people count embedding         vector(1536)  -- text-embedding-
3-small cluster_id        UUID REFERENCES issue_clusters(id) ministry_mapped   TEXT  --
responsible ministry created_at        TIMESTAMPTZ DEFAULT NOW()
```

issue_clusters

```
id               UUID PRIMARY KEY DEFAULT gen_random_uuid() constituency_id   INTEGER
REFERENCES constituencies(id) label             TEXT  -- cluster human-readable label
category          TEXT  -- road|water|power|health|education|employment issue_count
INTEGER DEFAULT 0 severity_avg      DECIMAL(3,1) velocity          DECIMAL  -- % change
week-over-week, positive=rising badge             TEXT  -- tatkal|rising|chronic|stable|
resolved first_seen        TIMESTAMPTZ DEFAULT NOW() last_updated      TIMESTAMPTZ
DEFAULT NOW() national_flag     BOOLEAN DEFAULT false  -- true if same cluster in 10+
constituencies
```

cluster_snapshots (weekly velocity baseline)

```
id               UUID PRIMARY KEY DEFAULT gen_random_uuid() cluster_id        UUID
REFERENCES issue_clusters(id) snapshot_date     DATE NOT NULL issue_count       INTEGER
severity_avg      DECIMAL(3,1) UNIQUE(cluster_id, snapshot_date)
```

parliamentary_actions

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid() constituency_id INTEGER
REFERENCES constituencies(id) cluster_id UUID REFERENCES issue_clusters(id)
action_type TEXT -- question_starred|question_unstarred|zero_hour|debate|pmb|
cut_motion|calling_attention content TEXT NOT NULL -- full draft text
source_citations JSONB -- [{title, url, type, date}] ministry TEXT
lok_sabha_rule TEXT -- e.g. "Rule 32" status TEXT DEFAULT 'draft' --
draft|submitted_to_mp|filed|response_received mp_approved BOOLEAN DEFAULT false
filed_at TIMESTAMPTZ session_reference TEXT -- e.g. "Winter Session 2025, Q No.
847" response_text TEXT response_received TIMESTAMPTZ created_at TIMESTAMPTZ
DEFAULT NOW()
```

mp_briefs

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid() constituency_id INTEGER
REFERENCES constituencies(id) week_date DATE NOT NULL -- Friday of the week
brief_content JSONB -- structured brief data brief_pdf_url TEXT -- S3/R2 URL
delivery_status TEXT -- pending|sent|delivered|failed delivered_at TIMESTAMPTZ
mp_whatsapp TEXT -- encrypted storage UNIQUE(constituency_id, week_date)
```

podcast_episodes (Phase 2 — create table but do not populate in Phase 1)

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid() week_date DATE NOT
NULL script_text TEXT audio_url TEXT video_url TEXT
fact_check_report JSONB -- {total_claims, verified, rewritten, removed} perspective_data
JSONB -- 6-perspective research output published_at TIMESTAMPTZ created_at
TIMESTAMPTZ DEFAULT NOW()
```

dissent_records

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid() citizen_id UUID
REFERENCES citizens(id) action_id UUID REFERENCES parliamentary_actions(id)
objection_text TEXT NOT NULL status TEXT DEFAULT 'open' -- open|reviewed|
accepted|rejected resolution_text TEXT created_at TIMESTAMPTZ DEFAULT NOW()
```

agent_logs (immutable audit trail — insert only, never update or delete)

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid() agent_type TEXT NOT
NULL -- intake|clustering|question_draft|mp_brief|fact_check|distribution
constituency_id INTEGER action TEXT NOT NULL -- what the agent did
input_hash TEXT NOT NULL -- SHA-256 of input data output_hash TEXT NOT NULL
-- SHA-256 of output data model_version TEXT NOT NULL -- e.g. claude-haiku-4-5
tokens_used INTEGER latency_ms INTEGER error_code TEXT -- null if
success created_at TIMESTAMPTZ DEFAULT NOW()
```

blockchain_anchors

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid() content_type TEXT --
episode|brief|action|session content_id UUID content_hash TEXT NOT NULL --
SHA-256 of content tx_hash TEXT -- Polygon transaction hash block_number
BIGINT anchored_at TIMESTAMPTZ takedown_notice BOOLEAN DEFAULT false -- true if
content deleted per legal order
```

3.2 Critical Indexes

```
-- Vector similarity search (pgvector) CREATE INDEX ON issues USING ivfflat (embedding
vector_cosine_ops) WITH (lists = 100); -- Constituency + type queries (most common
dashboard query) CREATE INDEX ON issues (constituency_id, issue_type, created_at DESC);
-- Cluster lookup CREATE INDEX ON issues (cluster_id); CREATE INDEX ON issue_clusters
```

```
(constituency_id, badge);  -- Audit trail queries CREATE INDEX ON agent_logs (agent_type, created_at DESC); CREATE INDEX ON agent_logs (constituency_id, created_at DESC);
```

3.3 Alembic Migration Order

1. Create constituencies table + seed data (543 rows)
2. Create citizens table
3. Create issues table (depends on citizens + constituencies)
4. Create issue_clusters table (depends on constituencies)
5. Add cluster_id FK to issues (after issue_clusters exists)
6. Create cluster_snapshots table
7. Create parliamentary_actions table
8. Create mp_briefs table
9. Create podcast_episodes table
10. Create dissent_records table
11. Create agent_logs table
12. Create blockchain_anchors table
13. Add pgvector extension + ivfflat index on issues.embedding

SECTION 4 — API ENDPOINTS

FastAPI. All responses are JSON. All endpoints use async/await. Authenticate with Bearer token where marked [AUTH]. Rate limit all public endpoints at 60 req/min/IP.

4.1 Public Endpoints (no auth required)

Method	Path	Description	Response shape
GET	/health	Health check — returns db + redis + agent status	{status, db, redis, agents_active, version}
GET	/api/constituency/{id}	Constituency metadata + current MP	{id, name, state, mp_name, mp_party, population}
GET	/api/constituency/{id}/issues	Top 10 clusters + full issue list with pagination	{clusters: [{label, count, severity, badge, velocity}], total, page}
GET	/api/constituency/{id}/timeline	12-month issue velocity by category — for charts	{category, data: [{week, count, severity_avg}]}
GET	/api/constituency/{id}/actions	All parliamentary actions filed — public after filing	{actions: [{type, content, status, filed_at, response_text}]}
GET	/api/national/pulse	Top issues aggregated across all active constituencies	{issues: [{label, constituency_count, avg_severity, total_reports}]}
GET	/api/national/heatmap	Per-constituency severity scores for map overlay	{constituencies: [{id, lat, lng, severity_score, top_category}]}
GET	/api/audit/weekly	Latest weekly bias audit report	{week, geographic_balance, party_distribution, fact_check_stats}

4.2 Citizen Endpoints

Method	Path	Description	Notes
POST	/api/citizen/submit	Submit issue via web portal — same pipeline as WhatsApp	Body: {text, constituency_id, language?, location?}; Requires Aadhaar OTP pre-verification
POST	/api/citizen/verify	Initiate Aadhaar OTP	Body:

		verification for constituency linking	{mobile_e164, constituency_id}; Returns session_token
POST	/api/citizen/verify/confirm	Confirm OTP and link citizen to constituency	Body: {session_token, otp}; Returns citizen_id + JWT
GET	/api/citizen/issue/{issue_id}	Check status of a submitted issue [AUTH — own issues only]	Returns cluster membership + rank + parliamentary action if any
POST	/api/citizen/dissent	File dissent against an agent parliamentary action [AUTH]	Body: {action_id, objection_text}; Returns dissent_id
DELETE	/api/citizen/data	DPDP right to erasure — delete all personal data [AUTH]	Deletes citizen record; anonymises issue text; keeps aggregate cluster data

4.3 WhatsApp Webhook

Method	Path	Description	Critical notes
POST	/webhook/whatsapp	Meta Cloud API webhook receiver	Verify webhook with WEBHOOK_VERIFY_TOKEN env var; Handle: text messages, voice notes (audio/ogg), status updates
GET	/webhook/whatsapp	Meta webhook verification challenge	Returns hub.challenge when hub.verify_token matches

```
# WhatsApp message handler flow: # 1. Receive payload → verify signature (X-Hub-Signature-256) # 2. Extract message type: text | audio | image # 3. If audio: download from Meta CDN → Whisper transcription # 4. Strip PII from raw payload before logging # 5. Queue to Celery task: process_citizen_issue.delay(payload) # 6. Send acknowledgment reply immediately (before Celery processes) # 7. Return 200 OK to Meta within 5 seconds (or Meta retries)
```

4.4 Authenticated Internal Endpoints

Method	Path	Auth level	Description
GET	/api/mp/brief/{constituency_id}	MP JWT — own constituency only	Current week brief + last 4 weeks history
GET	/api/mp/actions/{constituency_id}	MP JWT — own constituency only	All draft parliamentary actions for their constituency
POST	/api/mp/action/{action_id}/approve	MP JWT — own constituency only	Mark action as MP-approved; triggers filing workflow

GET	/api/journalist/package/{week_date}	Journalist API key	Weekly press package: story angles + structured data + embeddable charts
GET	/api/journalist/constituency/{id}/data	Journalist API key	Full constituency intelligence: all clusters, trends, parliamentary actions
GET	/api/admin/agents/health	Admin JWT	Health status of all agent instances with last-run timestamps
POST	/api/admin/pipeline/pause	Admin JWT + documented reason	Emergency pipeline kill-switch — logs reason + timestamp

SECTION 5 — AGENT SPECIFICATIONS

Each agent is a Python class with a run() method. All agents log to agent_logs. All agents use the model specified — do not substitute. Agent prompts are in /app/agents/prompts/. Keep prompts in separate .txt files — never hardcode in Python.

5.1 Intake Agent

Property	Specification
Class	IntakeAgent in app/agents/intake.py
Model	claude-haiku-4-5 (fast + cheap — this runs on every submission)
Trigger	Every WhatsApp message + web form submission — real-time, not scheduled
Input	Raw message text (already transcribed if voice note) + metadata
Output	Structured JSON: {issue_type, severity_score, location_district, location_ward, urgency_flag, affected_estimate, ministry_mapped}

```
# Extraction prompt must instruct model to: # - Return ONLY valid JSON, no preamble # - issue_type: one of [road, water, power, health, education, employment, housing, environment, other] # - severity_score: float 1.0-10.0 (1=minor inconvenience, 10=life-threatening emergency) # - urgency_flag: true only if issue poses immediate safety risk # - affected_estimate: integer estimate of people affected (not families — people) # - ministry_mapped: string name of responsible central ministry # - If field cannot be determined reliably, return null — never hallucinate
```

```
After extraction: generate embedding via openai text-embedding-3-small (1536 dims). Store embedding in issues.embedding column. This is what enables semantic cluster matching.
```

5.2 Clustering Agent

Property	Specification
Class	ClusteringAgent in app/agents/clustering.py
Model	No LLM — scikit-learn BERTopic + DBSCAN on stored embeddings
Trigger	Celery beat schedule — every 15 minutes
Input	All embeddings for a constituency from the last 7 days
Output	Updated issue_clusters table + badge assignments + Top 10 ledger

```
# Clustering logic: # 1. Load all embeddings for constituency from last 7 days # 2. Run DBSCAN(eps=0.18, min_samples=5) on embeddings # 3. For each cluster: compute centroid, count members, avg severity # 4. Generate cluster label: use Claude Haiku on 5 representative issue texts # 5. Calculate velocity: compare this week count vs last week snapshot # 6. Assign badge: # - tatkal: severity_avg >= 8.0 AND cluster_age < 24h AND new_reports > 50 # - rising: velocity >= +25% # - chronic: cluster_age > 30 days AND status != resolved # - resolved: government acknowledgment received (manual flag for now) # - stable: everything else # 7. Minimum cluster size: 5 issues before creating cluster # 8. Minimum cluster size: 50 issues before publishing to public dashboard
```

5.3 Question Hour Draft Agent

Property	Specification
Class	QuestionDraftAgent in app/agents/question_draft.py
Model	claude-sonnet-4-6 (quality matters — these are parliamentary instruments)
Trigger	Celery beat — daily at 07:00 IST
Input	Top 3 clusters per constituency from Clustering Agent
Output	Draft parliamentary questions stored in parliamentary_actions table

```
# Question drafting rules (encode in system prompt): # MANDATORY ELEMENTS in every question: # 1. Opening: 'Will the Minister of [Ministry] be pleased to state:' # 2. Sub-parts (a)(b)(c)(d) — minimum 3 sub-parts per question # 3. Constituency evidence: 'X verified citizens from [Constituency] have reported...' # 4. Statutory hook: cite CAG report / RTI response / scheme guideline / ministry's own record # 5. Specific ask: named project / contract / scheme / timeline — not vague generalities # 6. Rule citation: 'To be raised under Rule 32 of Lok Sabha Rules of Procedure' # # HARD GATE — question CANNOT be finalised if: # - Any factual claim lacks a primary source citation # - Ministry mapping is uncertain (mark as needs_review instead) # - Constituency evidence cites fewer than 10 verified reports # # Output format: {question_text, question_type (starred/unstarred), ministry, # lok_sabha_rule, source_citations: [{title,url,type,date}], # citizen_count, cluster_id, confidence_score}
```

5.4 MP Brief Generator

Property	Specification
Class	MPBriefAgent in app/agents/mp_brief.py
Model	claude-sonnet-4-6
Trigger	Celery beat — every Friday 08:00 IST
Input	Top 5 clusters + 5 best draft questions for constituency
Output	Structured JSON brief + PDF rendered via WeasyPrint + WhatsApp delivery

```
# Brief structure (1 page PDF): # Header: AgentSabha Co-Pilot | [Constituency Name] | Week of [date] # Section 1: TOP 5 ISSUES THIS WEEK # - Issue name | Category | Reports this week | Severity | Badge # Section 2: 5 DRAFT PARLIAMENTARY QUESTIONS # - Each with: ministry, rule, sub-parts, source citations # Section 3: TATKAL ALERT (if any — red box) # Section 4: CONSTITUENCY VS NATIONAL AVERAGE # - How does this constituency compare on top issue categories # Footer: 'Review and file any question at your discretion. AgentSabha | agentsabha.in' # # IMPORTANT: Brief must be readable in under 2 minutes. No walls of text. # WhatsApp delivery: send PDF + 3-line summary text message # Delivery confirmed via Meta webhook status update
```

5.5 Fact Check Agent (Phase 2 — media pipeline)

Property	Specification
Class	FactCheckAgent in app/agents/fact_check.py
Model	claude-sonnet-4-6 + web_search tool enabled

Trigger	After Podcast Script Agent — HARD GATE before any publish
Input	Full podcast script text
Output	Verified script + fact_check_report JSON — pipeline BLOCKED if unverified claims remain

```
# Fact check process: # 1. Parse every factual claim (number, date, name, attribution, statistic) # 2. For each claim: search citation database from Perspective Research Agent # 3. Classify: VERIFIED | UNCERTAIN (paraphrase mismatch) | UNVERIFIED (no source) # 4. UNCERTAIN: rewrite claim to match source exactly, mark as rewritten # 5. UNVERIFIED: remove claim from script entirely # 6. HARD GATE: if any UNVERIFIED claims remain after 3 search attempts → BLOCK publish # 7. Generate report: {total, verified, rewritten, removed, sources_list} # 8. Store report in podcast_episodes.fact_check_report # ZERO TOLERANCE: no unverified claims ever published
```

SECTION 6 — CELERY TASKS & PIPELINE ORCHESTRATION

All async work runs via Celery with Redis as broker and result backend. Tasks are in app/tasks/. Beat schedule in app/celery_config.py.

6.1 Task Definitions

Task name	Queue	Schedule	Description
process_citizen_issue	intake	On event (real-time)	Transcribe → translate → extract → embed → store. Called from WhatsApp webhook.
run_clustering	clustering	Every 15 minutes	BERTopic on each active constituency. Updates clusters + badges + Top 10 ledger.
take_cluster_snapshot	snapshots	Every Sunday 00:00 IST	Snapshot cluster state for velocity calculation next week.
generate_draft_questions	questions	Daily 07:00 IST	Question Draft Agent on top 3 clusters per constituency.
generate_mp_briefs	briefs	Friday 08:00 IST	MP Brief Agent for all constituencies with registered MP WhatsApp.
deliver_mp_briefs	delivery	Friday 09:00 IST	WhatsApp Business API send for all generated briefs.
check_parliamentary_responses	monitor	Daily 10:00 IST	Poll Lok Sabha website/API for responses to filed questions.
send_citizen_notifications	notify	On event (after response received)	Notify citizens whose cluster triggered a filed question.
run_weekly_audit	audit	Friday 20:00 IST	Generate + publish weekly bias audit report.
media_pipeline_trigger	media	Friday 00:00 IST (Phase 2)	Trigger full automated podcast/reels pipeline.

6.2 Celery Beat Configuration

```
# app/celery_config.py CELERYBEAT_SCHEDULE = {     'clustering-every-15-min': {         'task': 'app.tasks.run_clustering',         'schedule': crontab(minute='*/15'),     },     'cluster-snapshot-weekly': {         'task': 'app.tasks.take_cluster_snapshot',         'schedule': crontab(day_of_week='sunday', hour=0, minute=0),     },     'draft-questions-daily': {         'task': 'app.tasks.generate_draft_questions',         'schedule': crontab(hour=7, minute=0),  # IST = UTC+5:30 → 01:30 UTC     },     'mp-brief-generation': {         'task': 'app.tasks.generate_mp_briefs',         'schedule': crontab(day_of_week='friday', hour=2, minute=30),  # 08:00 IST     },     'mp-brief-delivery': {         'task': 'app.tasks.deliver_mp_briefs',         'schedule': crontab(day_of_week='friday', hour=3, minute=30),  # 09:00 IST     },     'parliamentary-response-monitor': {         'task': 'app.tasks.check_parliamentary_responses',         'schedule': crontab(hour=4, minute=30),  # 10:00 IST     },     'weekly-audit': {         'task': 'app.tasks.run_weekly_audit',         'schedule': crontab(day_of_week='friday', hour=14, minute=30),  # 20:00 IST     }, }
```

SECTION 7 — PROJECT STRUCTURE

```
agentsabha/ ├── backend/ │   ├── app/ │   │   ├── main.py              # FastAPI app factory │   │   ├── config.py            # Settings via pydantic-settings │   │   ├── database.py          # Async SQLAlchemy engine + session │   │   ├── models/ # SQLAlchemy ORM models (one file per table) │   │   │   ├── constituency.py │   │   │   ├── citizen.py │   │   │   ├── issue.py │   │   │   ├── cluster.py │   │   │   ├── parliamentary_action.py │   │   │   ├── mp_brief.py │   │   │   ├── agent_log.py │   │   │   └── blockchain_anchor.py │   │   ├── routers/             # FastAPI routers (one per domain) │   │   │   ├── public.py        # /api/constituency, /api/national │   │   │   ├── citizen.py       # /api/citizen │   │   │   ├── mp.py            # /api/mp  [AUTH] │   │   │   ├── journalist.py    # /api/journalist  [AUTH] │   │   │   ├── webhook.py       # /webhook/whatsapp │   │   │   └── admin.py         # /api/admin  [AUTH] │   │   ├── agents/ │   │   │   ├── intake.py        # IntakeAgent │   │   │   ├── clustering.py    # ClusteringAgent │   │   │   ├── question_draft.py# QuestionDraftAgent │   │   │   ├── mp_brief.py      # MPBriefAgent │   │   │   ├── fact_check.py    # FactCheckAgent (Phase 2) │   │   │   ├── voice_synth.py   # VoiceSynthAgent (Phase 2) │   │   │   └── prompts/ # .txt prompt files — one per agent │   │   │       ├── intake_extract.txt │   │   │       ├── question_draft_starred.txt │   │   │       ├── question_draft_unstarred.txt │   │   │       ├── mp_brief_summary.txt │   │   │       └── cluster_label.txt │   │   ├── tasks/ │   │   │   ├── intake.py        # process_citizen_issue task │   │   │   ├── clustering.py    # run_clustering task │   │   │   ├── questions.py     # generate_draft_questions task │   │   │   ├── briefs.py        # generate_mp_briefs + deliver_mp_briefs │   │   │   ├── monitor.py       # check_parliamentary_responses │   │   │   └── audit.py         # run_weekly_audit │   │   ├── services/ │   │   │   ├── whatsapp.py      # Meta Cloud API client │   │   │   ├── aadhaar.py       # UIDAI OTP client │   │   │   ├── embedding.py # OpenAI embeddings client │   │   │   ├── translation.py   # IndicTrans2 client │   │   │   ├── transcription.py # Whisper client │   │   │   └── pdf_generator.py # WeasyPrint brief PDF │   │   └── utils/ │   │       ├── hashing.py       # SHA-256 + PII hashing │   │       ├── encryption.py    # AES-256 for Tier 0 data │   │       └── audit_logger.py  # Structured agent_logs writer │   ├── alembic/               # Database migrations │   ├── tests/ │   │   ├── test_intake.py │   │   ├── test_clustering.py │   │   ├── test_question_draft.py │   │   └── test_full_loop.py    # End-to-end loop test │   ├── celery_worker.py │   ├── celery_config.py │   ├── requirements.txt │   └── Dockerfile ├── frontend/ │   ├── app/                    # Next.js 14 app directory │   │   ├── page.tsx            # Homepage with India map │   │   ├── constituency/[id]/   # Constituency dashboard │   │   └── api/                # Next.js API routes (proxy to FastAPI) │   ├── components/ │   │   ├── IndiaMap.tsx         # TopoJSON India map with heat overlay │   │   ├── IssueLedger.tsx      # Top 10 issues table with badges │   │   ├── SeverityBar.tsx      # Category severity bar chart │   │   ├── TrendBadge.tsx       # Rising/Tatkal/Chronic/Stable badge │   │   └── QuestionLog.tsx      # Draft questions ledger │   ├── lib/ │   │   └── api.ts               # Typed API client │   └── package.json ├── docker-compose.yml ├── .env.example └── README.md
```

7.1 Environment Variables (.env.example)

```
# Database DATABASE_URL=postgresql+asyncpg://user:pass@localhost:5432/agentsabha REDIS_URL=redis://localhost:6379/0  # AI / LLM ANTHROPIC_API_KEY=sk-ant-... OPENAI_API_KEY=sk-...           # for text-embedding-3-small only  # WhatsApp Meta Cloud API WHATSAPP_TOKEN=EAAxxxxxxxx WHATSAPP_PHONE_ID=1234567890 WHATSAPP_VERIFY_TOKEN=your-random-verify-token WHATSAPP_APP_SECRET=xxxxxxxx   # for signature verification  # Aadhaar UIDAI (sandbox for Phase 1) UIDAI_AUA_CODE=your-aua-code UIDAI_ASA_CODE=your-asa-code UIDAI_LICENSE_KEY=your-license-key  # Encryption (AES-256 for Tier 0 data) ENCRYPTION_KEY=generate-32-byte-base64-key  # Storage (India region) AWS_REGION=ap-south-1 AWS_BUCKET_NAME=agentsabha-briefs AWS_ACCESS_KEY_ID=AKIA... AWS_SECRET_ACCESS_KEY=... # App settings ENVIRONMENT=development          # development | staging | production SECRET_KEY=generate-random-64-char-string CORRECT_CONSTITUENCY_MINIMUM_CLUSTER_SIZE=50  # min issues before publishing cluster TATKAL_SEVERITY_THRESHOLD=8.0 TATKAL_NEW_REPORTS_THRESHOLD=50 TATKAL_MAX_AGE_HOURS=24
```

SECTION 8 — BUILD ORDER (STRICT SEQUENCE)

> *Do not skip steps. Do not reorder steps. Each step builds on the previous. At the end of each week, the Definition of Done must be fully met before proceeding.*

Week 1–2: Foundation & Data Layer

Task	Definition of Done	Verify with
PostgreSQL 16 + pgvector extension running in Docker	psql connects; SELECT vector_dims('[1,2,3]'::vector) returns 3	docker-compose up; manual psql check
All 12 Alembic migrations run successfully (in order from Section 3.3)	alembic upgrade head completes; all tables exist; seed data loaded	alembic current shows head; SELECT count(*) FROM constituencies returns 543
FastAPI skeleton: /health endpoint returns 200 with db+redis status	GET /health returns {status: ok, db: ok, redis: ok}	pytest tests/test_health.py
Redis + Celery worker running and receiving test tasks	Celery worker logs show task received and executed	celery -A celery_worker inspect active
WhatsApp webhook: verification challenge response works	Meta Platform webhook verification passes	Use Meta Developer Console webhook tester
Aadhaar OTP: sandbox integration returns test VID token	OTP flow completes in sandbox; VID stored (not Aadhaar number)	Manual test in UIDAI sandbox environment

Week 3–4: Intake Agent

Task	Definition of Done	Test command
WhatsApp text message received → stored in issues table	POST to /webhook/whatsapp with text payload → issue row created in DB	Send test WhatsApp from real phone; verify DB row
Voice note received → Whisper transcription → stored	Audio ogg downloaded from Meta CDN → transcribed → stored with source_language	Send voice note in Hindi; verify transcription quality
IndicTrans2 translation: 5 test languages	Tamil, Malayalam, Kannada, Hindi, Telugu text correctly translated to English	pytest tests/test_translation.py with gold-standard translations
Claude Haiku extraction: all 8 fields returned as valid JSON	100 synthetic test issues → all 8 fields extracted with no null hallucinations	pytest tests/test_intake.py -v (min 95% field accuracy)
Embedding generated + stored in pgvector	issues.embedding column populated for all test issues; cosine similarity search returns semantically relevant results	SELECT id FROM issues ORDER BY embedding <=> query_embedding LIMIT 5 — manual review
Citizen acknowledgment	WhatsApp reply received with	End-to-end manual test with

sent within 60 seconds	issue_id and constituency rank	real phone

Week 5: Clustering Agent

Task	Definition of Done	Test
BERTopic pipeline runs on 200 seeded issues per constituency	Clusters generated; each issue has cluster_id; cluster labels make semantic sense	Seed test data; run task; manual review of 5 clusters per constituency
Velocity calculation correct	Take cluster snapshot; inject 50 new similar issues; run clustering; verify velocity = +50%	pytest tests/test_clustering.py::test_velocity_calculation
Badge assignment logic	Tatkal: inject 60 issues severity 9.0 in <24h → badge=tatkal. Chronic: set first_seen 45 days ago → badge=chronic	pytest tests/test_clustering.py::test_badge_assignment
Top 10 ledger API returns correct data	GET /api/constituency/1/issues returns 10 clusters sorted by severity_avg DESC	pytest tests/test_api.py::test_constituency_issues
Minimum cluster size: <50 issues not returned in public API	Create cluster with 40 issues; verify not returned in /api/constituency/{id}/issues	pytest tests/test_api.py::test_minimum_cluster_size

Week 6: Question Draft Agent

Task	Definition of Done	Quality gate
Ministry mapping lookup: all issue_types mapped	All 9 issue_types map to correct central ministry; mapping stored in DB	Manual review: road→MoRTH, water→MoJS, power→MoP, health→MoHFW, education→MoE
Lok Sabha rules RAG: Rule 32-55 loaded and queryable	Agent correctly identifies rule for starred vs unstarred question	pytest tests/test_question_draft.py::test_rule_citation
Starred question output: all mandatory elements present	Question text contains: Minister opening, 4 sub-parts (a)(b)(c)(d), constituency evidence (min 10 reports), statutory hook, source citations, Rule 32 citation	Human review of 10 generated questions: score ≥8/10 'could be filed by MP'
HARD GATE: unverified claims removed	Inject question data with one claim having no source → claim removed from output; gate logs removal	pytest tests/test_question_draft.py::test_source_gate
Draft question stored in parliamentary_actions table	Generated questions in DB with status=draft, source_citations JSONB populated	SELECT count(*) FROM parliamentary_actions WHERE status='draft' — must be > 0

Week 7: MP Brief + Dashboard

Task	Definition of Done	Test

Weekly PDF brief renders correctly (1 page)	PDF opens; all 5 sections present; readable in 2 min	Generate brief for test constituency; open PDF; time reading
WhatsApp delivery confirmed	Brief PDF + summary message delivered to test phone number; delivery status updated in mp_briefs table	End-to-end with real phone number
Public dashboard: constituency map renders	Next.js page loads; India TopoJSON map shows 3 constituencies highlighted; no JS errors in console	Open in Chrome Android + Safari iOS — both must render
Issue ledger: updates within 15 min of new submission	Submit test issue; verify appears in constituency ledger within 15 minutes	Manual timing test
Citizen web form: submits to same pipeline as WhatsApp	Submit via web form; verify same issue structure created in DB; same acknowledgment flow	pytest tests/test_full_loop.py::test_web_form_submission

Week 8: Integration + Security + MP Onboarding

Task	Definition of Done	Gate
End-to-end loop test: all 8 steps complete	Submit 10 synthetic citizen issues across 3 constituencies; full loop completes within SLA for all 10; mock MP approval triggers notification	pytest tests/test_full_loop.py — all 10 scenarios pass
Load test: 15-min clustering SLA under load	Inject 1,000 issues in 60 seconds; clustering pipeline completes within 15 min	Locust or k6 load test; monitor Celery worker logs
Security checklist: Tier 0-1 data protection	(1) No Aadhaar number in any DB column. (2) mobile_hash is SHA-256 not plaintext. (3) No PII in application logs. (4) Aadhaar VID encrypted at rest. (5) Role-based access prevents MP A accessing MP B's data	Manual security audit against Section 2 checklist
MP co-pilot onboarding: test brief delivered	Real MP (or stand-in) received Friday brief on WhatsApp; acknowledged receipt	Confirmation from MP or staff contact
First real citizen submissions from target constituency	≥10 real citizen submissions (not synthetic) in DB from target constituency	SELECT count(*) FROM issues WHERE constituency_id = [target] -- use Thiruvananthapuram (id=1), Bengaluru South (id=2), or Gurugram (id=3) AND created_at > [onboarding date]

Phase 1 Exit Gates — ALL 8 must be true before Phase 2

#	Exit gate	Verified by
1	1 parliamentary question derived from AgentSabha citizen cluster data filed in real Lok Sabha session	Lok Sabha bulletin / session record with

		question number
2	Citizen whose cluster triggered the question received WhatsApp notification	WhatsApp delivery receipt + citizen screenshot or verbal confirmation
3	1 national publication story on the co-pilot model with full loop documented	Published article URL + journalist name
4	Public dashboard live with real data for 3 constituencies	Public URL accessible without login; data is real, not seeded
5	100% fact-check pass rate on all MP briefs produced	agent_logs: zero records with fact_check_failed=true
6	Third-party security audit completed; critical findings resolved	Security firm sign-off document
7	IT Rules 2021 + DPDP consent framework reviewed by legal counsel	Legal counsel memo or email sign-off
8	1 civil society organisation has formally endorsed in writing	Written endorsement document or press release

SECTION 9 — FRONTEND SPECIFICATIONS

Next.js 14 with App Router. TypeScript. Tailwind CSS. shadcn/ui for components. No custom CSS where Tailwind suffices. Mobile-first — India's majority web traffic is mobile.

9.1 Design System (from AgentSabha brand guidelines)

Token	Hex	Usage
--neela	#1B2A4A	Primary — headers, nav, dark rows, buttons
--kesariya	#FF9933	Accent only (~10% of screen) — CTAs, badges, highlights
--sindoori	#C8592A	Critical / Tatkal alerts, severity high, error states
--hariyali	#2D6A4F	Resolved, success, positive trends, verified
--haldi	#F5E6C8	Page background (~65% of screen)
--mitti	#A0522D	Chronic unresolved, secondary text on light bg
--haath-kagaz	#FAF3E0	Card surfaces, form backgrounds

> *NEVER use: gradients, glassmorphism, purple (#6366F1), rounded-full pill buttons as primary CTAs. Card border-radius MAX 3px — newspaper clipping feel. Category badge: rubber stamp style, 1.5px border, 3px radius.*

9.2 Typography

Role	Font	Weight	Notes
Display / Hero headings	Playfair Display	700	Import from Google Fonts
Hindi display	Tiro Devanagari	400	Equal visual weight to English — never secondary
Body / UI / buttons	DM Sans	400, 500	Clean, accessible
Data labels / badges	DM Sans	600	Letter-spacing: 0.04em
Citizen quotes	Playfair Display	400 italic	Used in podcast script + quote callouts

9.3 Pages to Build (Phase 1)

Route	Component	Key elements
/ (Homepage)	app/page.tsx	India map with 3 active constituencies highlighted; live ticker (category + location + timestamp scrolling); hero: '543 AI Agents' + active count; Submit Issue CTA
/constituency/[id]	app/constituency/[id]/page.tsx	Constituency name + MP name + population; Top 10 issue ledger with badges; Severity bar chart by category; Draft question log (latest 5); Submit issue button

/submit	app/submit/page.tsx	Issue submission form; language selector; WhatsApp QR code / link as primary option; web form as backup; Aadhaar OTP constituency verification
/about	app/about/page.tsx	How it works (3-step: citizen → AI → Parliament); Design principles; Data privacy policy summary

9.4 India Map Component (IndiaMap.tsx)

```
// Use: https://github.com/datameet/maps — official India constituency TopoJSON // npm install topojson-client d3  // Required behaviour: // - Render all 543 constituency polygons // - Fill colour based on severity_score from /api/national/heatmap // severity 1-3: var(--hariyali) at 20% opacity //   severity 4-6: var(--kesariya) at 30% opacity //   severity 7-8: var(--sindoori) at 40% opacity //   severity 9-10: var(--sindoori) at 70% opacity // - Active Phase 1 constituencies: add pulse ring animation // - Hover tooltip: constituency name + MP name + top issue + report count // - Click: navigate to /constituency/[id] // - Mobile: pinch-zoom enabled; double-tap to reset // - Min polygon size for tap: 44px (iOS tap target minimum)
```

9.5 Issue Ledger Component (IssueLedger.tsx)

```
// Props: { constituencies_id: number } // Data: GET /api/constituency/{id}/issues // Polling interval: 60 seconds (real-time feel without hammering API)  // Each row shows: // [TrendBadge] [Category] [Issue Label] [Report Count] [Severity] [Since]  // TrendBadge colours: // TATKAL: var(--sindoori) background — pulsing animation // RISING: var(--kesariya) background // CHRONIC: var(--mitti) background // STABLE: var(--neela) 20% opacity background // RESOLVED: var(--hariyali) background  // Minimum cluster size check: API already filters <50 — trust the API // Show skeleton loading state while fetching // Show 'Last updated: X minutes ago' timestamp
```

SECTION 10 — TESTING REQUIREMENTS

Write tests as you build. Not after. Tests are the proof that the loop works. The test suite is what enables handoff between developers without regressions.

10.1 Required Test Files

Test file	What it tests	Minimum coverage
tests/test_intake.py	Intake Agent extraction accuracy: issue_type, severity, location, urgency, ministry on 100 synthetic issues	95% field accuracy; 0% hallucinated fields
tests/test_clustering.py	BERTopic clustering, velocity calc, badge assignment, minimum cluster size enforcement	All badge types triggered correctly; velocity math verified
tests/test_question_draft.py	Question structure, mandatory elements, HARD GATE for unsourced claims, ministry mapping	100% of questions have all mandatory elements; 0% unsourced claims pass gate
tests/test_api.py	All public API endpoints return correct shape; auth gates enforced; minimum cluster size respected	All endpoints return correct HTTP codes; 401 on protected endpoints without auth
tests/test_whatsapp.py	Webhook signature verification; message routing; acknowledgment sent within SLA	Forged signatures rejected; SLA met in 95% of test runs
tests/test_security.py	No PII in logs; Aadhaar not stored; mobile_hash is SHA-256; Tier 0 data encrypted	All checks pass before any production deploy
tests/test_full_loop.py	Complete end-to-end: citizen submit → cluster → question draft → brief → mock approval → notification	All 8 steps complete; all SLAs met; audit log populated correctly

10.2 Golden Test Dataset

Create tests/fixtures/golden_issues.json with 100 test issues covering:

- 20 road/infrastructure issues in Hindi and Kannada (various severity levels)
- 20 water supply issues in Malayalam and Tamil
- 15 health centre issues in Kannada and Malayalam
- 15 power supply issues in Hindi and Tamil
- 10 education issues in Gujarati and Odia
- 10 employment/MGNREGA issues in Rajasthani and Chhattisgarhi
- 5 voice notes (pre-transcribed, include original text + expected translation for each)
- 5 edge cases: abusive content, irrelevant content, empty message, very short message, duplicate of existing issue

10.3 Pre-Production Checklist

- All tests pass: pytest -v returns 0 failures
- Load test: 1,000 concurrent submissions processed within SLA
- Security checklist from Section 2.2 verified manually
- Aadhaar sandbox OTP flow working end-to-end

- WhatsApp webhook verified via Meta Developer Console
- All environment variables documented in .env.example
- Database migrations run cleanly from empty DB
- docker-compose up brings up full stack with no errors
- MP brief PDF renders correctly on both mobile and desktop PDF viewers
- Public dashboard renders on Chrome Android 90+ and Safari iOS 15+

SECTION 11 — SESSION HANDOFF PROTOCOL

Because any LLM has a context window limit, this section defines how to hand off between sessions without losing state or making inconsistent decisions.

> *Every LLM session working on AgentSabha MUST start by reading this document. Every session MUST end by updating the Session State below. No exceptions.*

11.1 At the Start of Every Session — Do This First

14. Read Sections 1, 2, and 3 of this document to restore context.
15. Read the most recent entry in SESSION_STATE.md in the repo root.
16. Read the last 20 lines of DECISIONS.md to understand recent choices.
17. Run: git log --oneline -10 to see what was last committed.
18. Run: pytest tests/ to see current test status.
19. ONLY THEN start writing code.

11.2 SESSION_STATE.md — Update at End of Every Session

> *# AgentSabha Session State Last updated: [ISO datetime] Last developer: [name or 'LLM session'] Current phase: Phase 1 Current week: Week [X] of 8 ## Completed this session - [List each task completed with DoD verified] ## In progress (DO NOT restart from scratch) - [Task name]: [current state and next step] ## Blocked - [Task name]: [blocker description and what is needed to unblock] ## Decisions made this session (add to DECISIONS.md) - [Decision]: [Reasoning] ## Next session must start with - [Specific task or test to run first] ## Known issues - [Any bugs found but not yet fixed]*

11.3 DECISIONS.md — Log Every Non-Obvious Decision

> *# AgentSabha Architecture Decisions ## [Date] — [Decision title] Context: [Why this decision needed to be made] Decision: [What was decided] Alternatives considered: [What else was evaluated] Reason: [Why this was chosen over alternatives] Consequences: [What this means for future work] --- ## Example entry: ## 2025-07-15 — BERTopic vs K-means for clustering Context: Needed clustering algorithm for issue grouping Decision: BERTopic + DBSCAN Alternatives: K-means, Agglomerative, LDA Reason: BERTopic does not require fixed cluster count; DBSCAN handles noise points; both work on embeddings directly without retraining Consequences: Cluster count varies each run; need to handle cluster merging across runs*

11.4 The One Question to Ask Before Any Code Change

> *'Does this change violate any constraint in Section 2 of the Master Prompt? If yes, stop and re-read Section 2 before proceeding.'*

11.5 If You Are Stuck or Unsure

- Re-read the relevant section of this document first.
- Check DECISIONS.md — the answer may already have been decided.
- Check SESSION_STATE.md — the context you need may be there.
- If still unclear: do NOT guess. Write a comment: # DECISION NEEDED: [question] and leave the code incomplete. Document in SESSION_STATE.md under 'Blocked'.
- Never hallucinate a business rule, schema column, API behaviour, or agent prompt. If it's not in this document, it has not been decided yet.

END OF MASTER BUILD PROMPT

"543 AI Agents. One Parliament. A Billion Voices."

agentsabha.in | Version 2.0 | March 2026 | CONFIDENTIAL